
From Python to Native C++

A Performance Study of Decoder-Only Transformers Using PyTorch and LibTorch

Om Suraj Kashikar

Grade 12

Silver Oaks International

Abstract

This study compares Python/PyTorch and C++/LibTorch implementations of a small decoder-only Transformer trained on TinyStories V2 GPT-4. Both implementations use the same GPT-2-tokenized training data, 256-token context length, and core Transformer architecture, allowing their training and inference performance to be evaluated on the same NVIDIA RTX 4090. The C++ implementation achieved higher observed training and inference throughput than Python, while also using less recorded memory in the benchmark. These results indicate performance and resource-efficiency benefits for the tested C++/LibTorch configuration, although the comparison is influenced by the different precision modes used: Python trained with mixed precision and gradient scaling, while C++ trained in FP32.

1 Introduction

Decoder-only Transformers are a widely used architecture for autoregressive language modeling. They predict the next token in a sequence while using causal self-attention to ensure that each token can attend only to earlier tokens. Modern language-model development is often performed through high-level Python frameworks such as PyTorch, while C++ can provide a lower-level interface through LibTorch, PyTorch's C++ API.

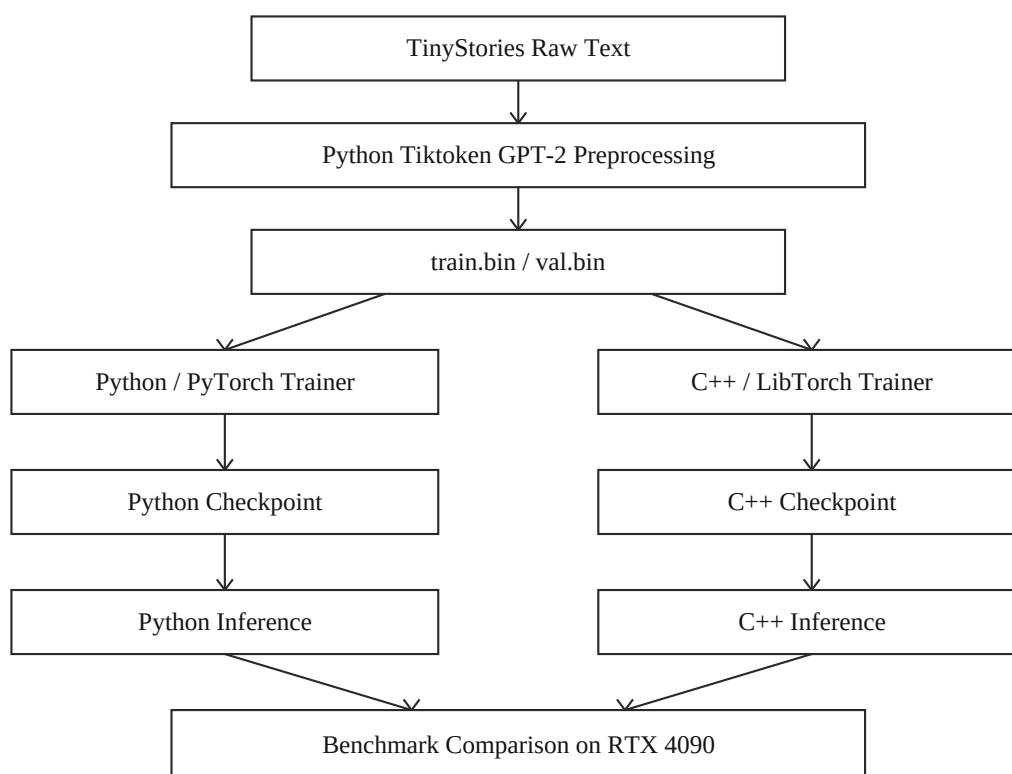
This project implements the same small GPT-style decoder-only Transformer in Python/PyTorch and C++/LibTorch. The objective is not to claim that one programming language is universally faster. Instead, the project evaluates two concrete implementations under a controlled model and data setup.

The central research question is:

How do Python/PyTorch and C++/LibTorch implementations of the same small decoder-only Transformer compare in training throughput, autoregressive inference throughput, and recorded memory use on an NVIDIA RTX 4090?

The work uses the TinyStories V2 GPT-4 corpus, a GPT-2-compatible vocabulary, identical pre-tokenized binary training data, and matching core model hyperparameters.

1.1 Workflow



2 Background

2.1 Transformer language model

A Transformer language model predicts the next token in a sequence by using the preceding tokens as context. A token may represent a complete word, part of a word, punctuation, or another text fragment. During training a sequence of input is shifted by one position to create prediction targets. For example

input sequence
The dog found a ball
To predict
predicted token
dog found a ball <next token>

The model is trained using cross-entropy loss over the vocabulary. A decoder-only Transformer is well suited to this task because it processes a sequence while applying a causal attention mask. The mask ensures that a token cannot access future tokens during training or

generation.

2.2 Self-Attention

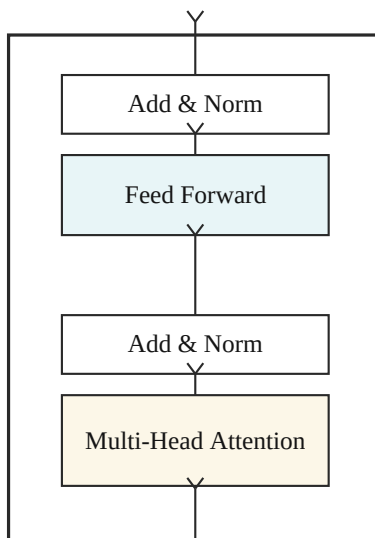
Self attention allows each token representation to look at previous tokens for information. For each head, the model produces queries, keys, and values. Attention scores are calculated by comparing queries with keys, scaled by the root of the head dimension, and normalized using softmax.

$$Attention(Q, K, V) = softmax((Q K^T) / sqrt(d_k)) V$$

The causal mask is applied on future positions before softmax. Multiple heads allow the model to learn different relationships between tokens. The outputs of each head are then concatenated and converted into the original embedding dimension.

2.3 Decoder-only transformer blocks

The model in this project uses pre-LayerNorm Transformer blocks. Each block contains causal multi-head self-attention and a feed-forward network, with residual connections around both components:



The feed-forward network uses a GELU activation and expands the hidden dimension before projecting it back to the embedding width.

$$GELU(x) = x \Phi(x)$$

where $\Phi(x)$ is the standard normal cumulative distribution function.

2.4 Python and C++ deep-learning APIs

Python is widely used for machine-learning research because it provides concise syntax and mature libraries. PyTorch's Python API provides automatic differentiation, CUDA execution, mixed precision, optimizers, checkpointing, and tensor operations.

LibTorch is PyTorch's C++ API. It provides similar tensor, autograd, CUDA, neural-network, and optimizer functionality through C++. Therefore, this project compares Python/PyTorch with C++/LibTorch rather than comparing Python with a completely framework-free implementation.

The C++ implementation is useful for studying the practical trade-offs of a compiled interface, including execution speed, memory behavior, project structure, and deployment requirements.

2.5 Mixed precision

Mixed-precision training uses lower-precision numerical formats for selected operations while retaining enough precision for stable optimization. It can improve speed and reduce memory use on compatible GPUs. The Python implementation in this project uses CUDA automatic mixed precision and gradient scaling. The completed C++ benchmark uses FP32 because an equivalent AMP-plus-gradient-scaling path is not implemented as it is out of the scope for this paper.

This precision difference is important when interpreting the benchmark. The results represent the tested Python/PyTorch and C++/LibTorch configurations, including their numerical precision choices.

```
170     betas=betas,
171 )
173 scaler = torch.amp.GradScaler('cuda', enabled=device.type == 'cuda')
175 train_data = Bindataset(train_bin, GPTConfig.block_size)
```

3 Dataset and tokenization

3.1 TinyStories V2 GPT-4

The model was trained on TinyStories V2 GPT-4, a dataset of short English stories generated by GPT-4. The simple vocabulary and story structure make it suitable for training and evaluating a small language model on limited hardware.

The dataset was divided into training and validation text files. The training data was used to update model parameters, while the validation data was used to monitor generalization during training.

3.2 Shared tokenization pipeline

The common .bin files were generated using GPT-2 tokenizer in the tiktoken library.

```
7     valid_file = "TinyStoriesV2-GPT4-valid.txt"
9     enc = tiktoken.get_encoding("gpt2")
11     def pretokenize_to_bin(filename, output_filename, chunk_size=10_000_000):
```

The token IDs were stored in two binary files:

```
-train.bin
-val.bin
```

Each token ID is stored as a uint16 value. Both the Python/PyTorch trainer and the C++/LibTorch trainer read these same files. The C++ training implementation did not tokenize the raw text independently.

3.3 Token data properties

Table 1: Tokenizer and Dataset Properties

Property	Value
Vocabulary	GPT-2
Vocabulary size	50,257
Storage type	uint16
Training file	train.bin
Validation file	val.bin
End-of-text token	ID 50,256

4 Model architecture

Both implementations use the same model architecture

Hyperparameter	Value
model type	decoder-only
Vocabulary size	50,257
context length	256
transformer blocks	8
attention heads	8
embedding dimension	384
attention head dimension	48
MLP hidden width	1536
positional embeddings	learned
Activation	GELU
normalization	Pre-LayerNorm
dropout	0.1
output weights	tied to token embeddings

4.1 Input representation

For an input tensor of token IDs with shape [B, T], the model obtains token embeddings and adds learned position embeddings. Here, B is the batch size and T is the sequence length, with

$$T \leq 256$$

4.2 Causal Self-Attention

The attention module projects the input into queries, keys, and values. The attention score matrix is calculated as:

$$Attention = softmax((Q K^T) / \sqrt{d_k})$$

Future positions are masked before softmax. The resulting attention weights are multiplied by the value representations to produce the attention output.

4.3 Feed-Forward Network

Each Transformer block contains a two-layer feed-forward network:

Linear(384 -> 1536)

GELU

Linear(1536 -> 384)

Dropout

4.4 Output projection and weight tying

The final hidden representation is normalized and projected to the 50,257-token vocabulary. The output projection uses the same weight matrix as the token embedding layer. This reduces duplicated parameters and follows the architecture used in the Python implementation.

5 Implementation details

5.1 Python implementation

The Python implementation uses PyTorch modules for embeddings, linear layers, LayerNorm, dropout, and causal attention, and the Transformer blocks. The training script uses AdamW, gradient clipping, learning-rate warmup, cosine decay, validation, and checkpoint saving.

During CUDA training, the Python implementation uses automatic mixed precision and GradScaler.

5.2 C++/LibTorch Implementation

To validate portability and enable deployment outside the Python ecosystem, we implement an equivalent training pipeline in C++ using LibTorch, the official C++ API for PyTorch. This implementation mirrors the Python reference architecture component-for-component, including the GPT configuration, causal self-attention, transformer blocks, token and position embeddings, layer normalization, GELU-activated MLP layers, cross-entropy loss, AdamW optimization, binary token-data loading, autoregressive generation, and checkpoint saving and loading.

Critically, the C++ trainer consumes the same train.bin and val.bin binary shards produced by the Python preprocessing pipeline, ensuring both implementations train on identical tokenized data. No separate tokenizer is implemented in the C++ path; tokenization remains the responsibility of the upstream Python data-preparation stage.

5.3 Weight tying

The Python model ties the language-model output weights to the token-embedding weights. The C++ implementation achieves the same behavior by directly using the token-embedding matrix for the output projection rather than registering an independent output parameter.

5.4 Precision

Table 2: Precisions configuration

Implementation	Record training Precision
Python/PyTorch	AMP/autocast with gradient scaling
C++/LibTorch	FP32

6 Experimental setup

6.1 Hardware

Both experiments were conducted on an NVIDIA RTX 4090.

Table 3: Hardware used

Hardware	Value
GPU	NVIDIA RTX 4090
CUDA version	13.4.0
Python version	3.14
GPU driver	595.99.02
LibTorch version	2.5.0
Cloud environment	Jupyter notebook

6.2 Training configuration

Table 4: Experimental setup for Training

Setting	Value
Batch size	32
context length	256
Tokens per step	8192
training steps	100,000
Nominal training tokens	819.2 million
Optimizer	AdamW
Learning rate	3e-4
Adam betas	(0.9, 0.95)
Weight decay	0.1
Bias/norm weight decay	0.0
Gradient clipping	1.0
Warmup steps	500
Learning rate schedule	cosine decay after warmup
Validation Interval	1000
Validation iterations	200

The tokens processed per step are:

$$32 * 256 = 8,192$$

The nominal total number of training tokens is:

$$100,000 * 8192 = 819,200,000$$

6.3 Inference configuration

Table 5: Experimental setup for text generation

Setting	Python	C++
Prompt	Same	Same
Context length	256	256
Maximum new tokens	256	256
Temperature	0.4	0.4
Top-k	50	50
Token selection	Argmax	Argmax
EOT stopping	Enabled	Enabled
GPU	RTX 4090	RTX 4090

The benchmark prompt was:

A small dog found a shiny ball

Because the models were trained independently, they can emit the EOT token at different points. Thus, actual output length may differ even though both systems use the same maximum generation cap.

7 Results

7.1 Training throughput

Table 6: Learning throughput rates

Implementation	Observed sustained throughput
Python/PyTorch	~59,000-62,000 toks/s
C++/LibTorch	~95,000-100,000 toks/s

Using representative midpoint values:

$$97,500 / 60,000 = 1.625$$

The C++ implementation achieved approximately 1.63x the Python training throughput, or approximately 62.5% higher throughput. Using the range boundaries, the observed relative advantage is approximately 1.56x to 1.69x.

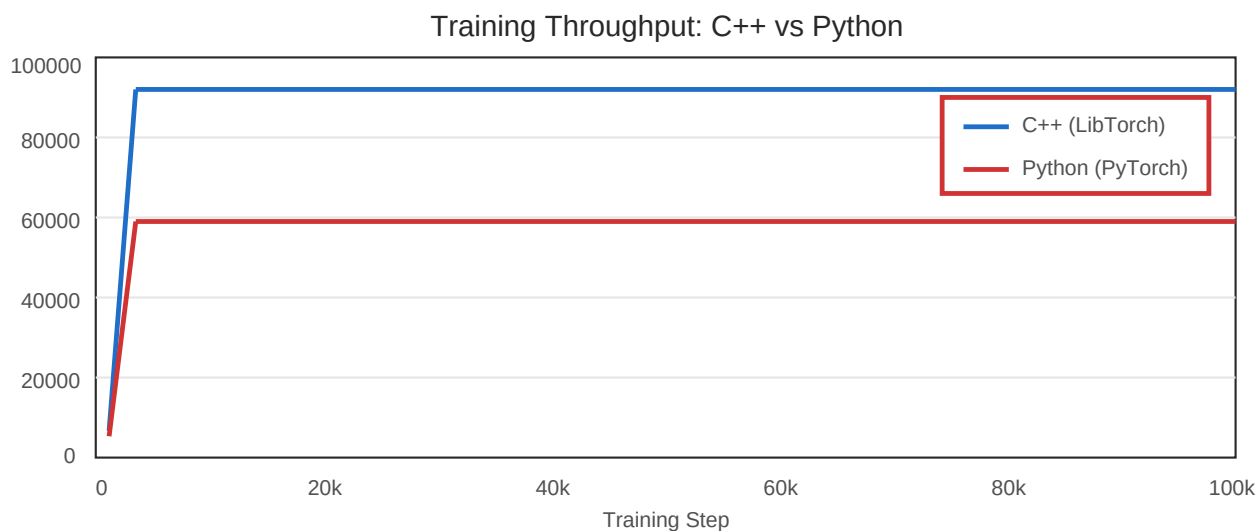


Figure 1: Training throughput (tokens/sec) over training steps for the C++/LibTorch and Python/PyTorch implementations.

7.2 Python validation results

The completed Python training run recorded the following validation losses (Table 7):

Table 7: Python (PyTorch) model validation loss at every 10,000 training steps

Step	Validation Loss
10,000	2.0457
20,000	1.7720
30,000	1.6515
40,000	1.5872
50,000	1.5276
60,000	1.4947
70,000	1.4601
80,000	1.4378
90,000	1.4194
99,000 (final)	1.4196

The best logged validation loss was 1.4088 at step 94,000. The final logged validation loss was 1.4196 at step 99,000. The full trajectory is shown in Figure 2.

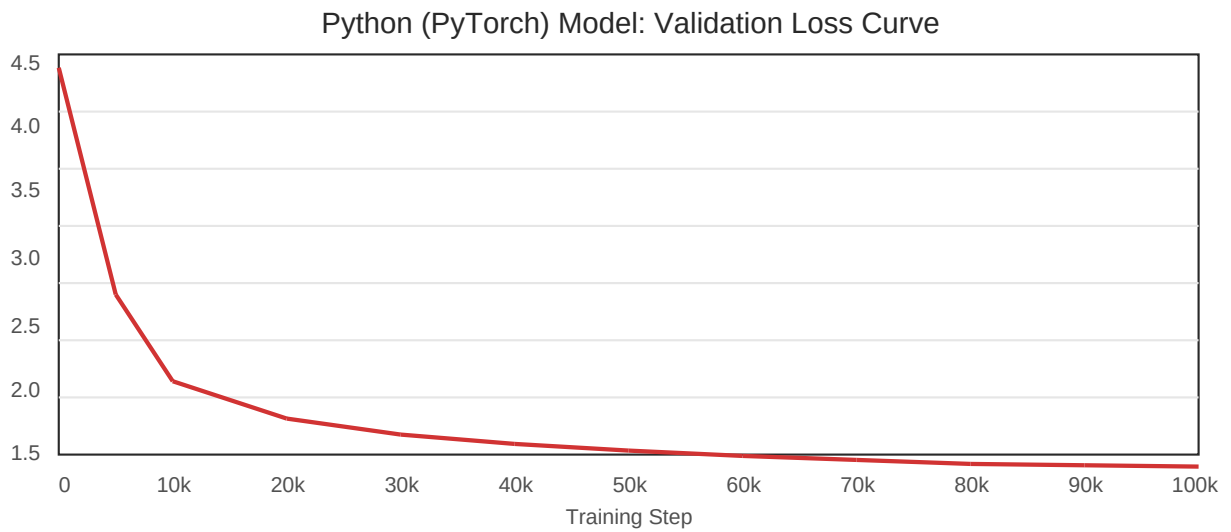


Figure 2: Validation loss over training steps for the Python (PyTorch) model.

7.3 C++ Validation Results

The completed C++ training run recorded the following validation losses (Table 8):

Table 8: C++ (LibTorch) model validation loss at every 10,000 training steps

Step	Validation Loss
10,000	2.0499
20,000	1.7788
30,000	1.6533
40,000	1.5860
50,000	1.5335
60,000	1.4971
70,000	1.4673
80,000	1.4374
90,000	1.4202
99,000 (final)	1.4136

The best logged validation loss was 1.4136, achieved at both step 95,000 and step 99,000. The final logged validation loss was 1.4136 at step 99,000. The full trajectory is shown in Figure 3.

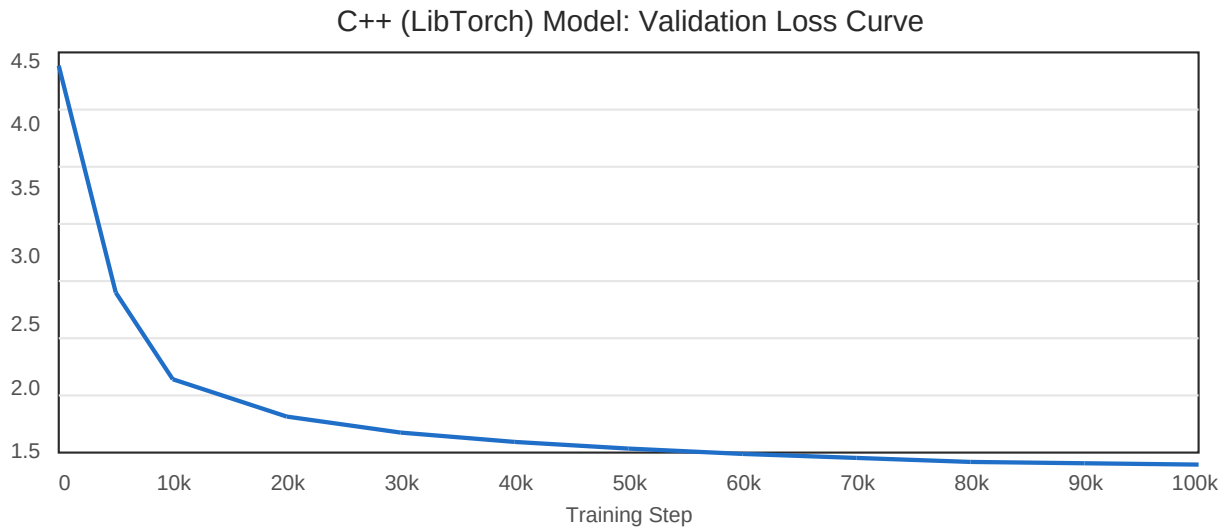


Figure 3: Validation loss over training steps for the C++ (LibTorch) model.

7.4 Inference throughput

Three inference runs were recorded for each implementation.

Table 9: Inference throughput and total time across multiple runs

Metric	C++ Run 1	C++ Run 2	C++ Run 3	Python Run 1	Python Run 2	Python Run 3
Throughput	190.6 tok/s	190.6 tok/s	192.254 tok/s	129 tok/s	123 tok/s	130 tok/s
Total time	0.666 s	0.666 s	0.659 s	0.61 s	0.62 s	0.61 s

Three inference runs were recorded for each implementation; the displayed throughput values are the raw recorded values, and the median is used as the summary statistic. The median throughput values are:

Table 10: Inference run median tok/s

Implementation	Median Inference throughput
Python/PyTorch	129.0 tok/s
C++/LibTorch	190.6 tok/s

The ratio is:

$$190.6 / 129 = 1.4775$$

Therefore, C++ achieved approximately 1.48x, or 47.8%, higher recorded inference throughput.

7.5 Memory results

Table 11: Inference GPU memory and System RAM across multiple runs

Metric	C++ Run 1	C++ Run 2	C++ Run 3	Python Run 1	Python Run 2	Python Run 3
System RAM	866.098 MB	864.7 MB	864.844 MB	1055 MB	1055 MB	1055 MB
GPU memory	164.78 MB	164.78 MB	164.79 MB	545 MB	545 MB	545 MB

Median values:

Table 12: Inference GPU memory and System RAM across multiple runs

Metric	C++	Python	difference
System RAM	864.7 MB	1055 MB	C++ approx 18% lower
GPU memory	164.78 MB	545 MB	C++ approx 69.8% lower

System-RAM reduction: $(1055 - 864.7) / 1055 * 100 \sim 18.0\%$

GPU memory reduction: $(545 - 164.78) / 545 * 100 \sim 69.8\%$

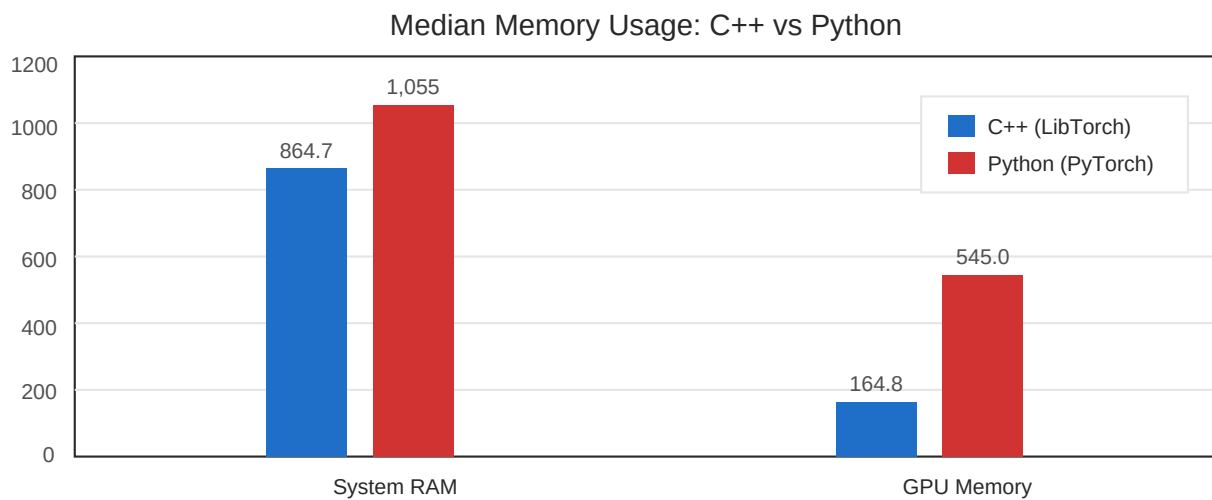


Figure 4: Median system RAM and GPU memory usage for the C++ and Python implementations.

8 Discussion

The measurements show that the C++/LibTorch implementation achieved higher observed training throughput than the Python/PyTorch implementation. C++ sustained approximately 95,000–100,000 training tokens/s, while Python sustained approximately 59,000–62,000 tokens/s. The C++ implementation also achieved higher recorded autoregressive inference throughput, with a median of 190.6 tok/s compared with 129 tok/s for Python.

The memory measurements also favored C++ in the recorded inference tests. C++ used approximately 864.7 MB of system RAM and 164.78 MB of GPU memory, compared with approximately 1,055 MB of system RAM and 545 MB of GPU memory for Python. These results should be interpreted carefully. The training data and model architecture were controlled well: both implementations used the same Python-generated token binaries, the same 256-token context length, the same 256-token maximum generation cap, and the same nominal architecture and optimization settings. However, the Python run used mixed precision and gradient scaling, whereas the C++ run used FP32. The benchmark therefore reflects the complete tested configurations rather than programming-language overhead alone.

The independently trained models can also emit EOT at different positions. This explains why their actual output lengths and total generation times can differ even when both are configured with the same context length and maximum generation cap. Tokens/sec is therefore the most useful primary inference metric for the current results.

9 Limitations

- This is a Python/PyTorch versus C++/LibTorch comparison, not a framework-free C++ CUDA comparison.
- Python training used AMP/autocast and GradScaler; C++ training used FP32.
- The models were trained independently, so their weights and outputs are not identical.
- Both systems use a 256-token context and a 256-token maximum generation cap, but EOT stopping can produce different actual output lengths.
- The inference benchmark contains three runs per implementation.
- Raw total inference time is not a strict equal-work latency measure when output lengths differ.
- The memory-measurement method and exact measurement point should be documented in the final paper.
- Python and C++ checkpoints use implementation-specific formats.
- Training-phase memory and inference-phase memory must be reported separately.

10 Conclusion

This study implemented and benchmarked the same small decoder-only Transformer in Python/PyTorch and C++/LibTorch. Both systems used the same Python-generated GPT-2 token binaries, the same 256-token context length, the same 256-token maximum generation cap, and the same core Transformer architecture.

On an NVIDIA RTX 4090, C++ sustained approximately 95,000–100,000 training tokens/s compared with approximately 59,000–62,000 tokens/s for Python. C++ also achieved a higher median inference throughput of 190.6 tok/s compared with 129 tok/s for Python. The C++ implementation recorded lower system RAM and GPU memory use in the inference measurements.

The results indicate that the tested C++/LibTorch implementation was more throughput-efficient under the measured conditions. However, the results are specific to the tested code and precision configurations. Python used AMP with gradient scaling, while C++ used FP32, so the findings should not be interpreted as proof that C++ is universally faster than Python.

References

- [1] E. Eldan and Y. Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? 2023. <https://arxiv.org/pdf/2305.07759>
- [2] A. Vaswani et al. Attention Is All You Need. 2017. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [3] A. Karpathy. Let us build GPT: from scratch, in code, spelled out. <https://youtu.be/kCc8FmEb1nY?si=5y37KxxpiBomNoQR>
- [4] A. Karpathy. Lecture on decoder-only transformers. <https://youtu.be/l8pRSuU81PU?si=2i59BjPM8fJaJtGE>
- [5] J. Starmer. StatQuest with Josh Starmer: Neural Networks and Transformers. <https://www.youtube.com/@statquest>
- [6] Race Engineering. Cloud GPU services. <https://raceengineering.ai>
- [7] Kashikar, Om Suraj Native C++ vs Python Decoder-Only Transformer. GitHub repository, 2026. https://github.com/25sh0363-code/Native_C-_vs_Python_decoder_only_model

Acknowledgments

This work draws on foundational material and educational resources, including the original Transformer architecture [2], Andrej Karpathy’s lectures on GPT-2 recreation and decoder-only transformers [3, 4], StatQuest’s explanations of neural network fundamentals [5], and cloud GPU infrastructure provided by Race Engineering [6]. The complete implementation is available at our GitHub repository [7].

Appendix A: Generated text samples

Python output

Prompt: A small dog found a shiny ball

Output:

A small dog found a shiny ball. The dog wanted the ball. The dog tried to get the ball, but it was too high. The dog was sad.

Then, a big bird came. The bird saw the dog and the shiny ball. The bird wanted to help. The bird flew up and got the ball for the dog. The dog was happy. The dog and the bird played with the shiny ball all day.

Tokens generated: 80
Time taken: 0.62 sec
Tokens/sec: 129.08
Peak RAM usage: 1055.78 MB
Peak GPU memory usage: 545.68 MB

C++ output

Prompt: A small dog found a shiny ball

A small dog found a shiny ball. The dog wanted to play with the ball. The dog ran to the ball and gave it a big push. The ball rolled and rolled. The dog was happy. The dog saw a big tree. The dog wanted to play with the ball. The dog pushed the ball with his nose. The ball rolled and rolled. The dog chased the ball. The dog was very happy. But then, the ball rolled into a hole. The dog tried to get the ball out, but it was too deep. The dog could not get the ball out. The dog was sad too. The end.

The samples are qualitative examples. Exact text matching is not expected because the models were trained independently.

Appendix B: Environment details

Item	Value
GPU	NVIDIA RTX 4090
CUDA version	13.4.0
NVIDIA driver	595.99.02
Python version	3.14
PyTorch version	2.14
LibTorch version	2.5.0
C++ compiler	GCC 11.4.0
CMake version	3.18
Cloud provider	Race Engineering [6]

Appendix C: Reproduction commands

Python

```
pip install -r requirements.txt
python3 tokenizer.py
python3 model.py
python3 test.py
```

C++

```
cmake -S . -B build -DCMAKE_PREFIX_PATH=/path/to/libtorch
cmake --build build --config Release -j2
./build/train
./build/generate
```

Appendix D: Final data summary

Table 13: Model architecture

Parameter	Value
Vocabulary	50,257
Context length	256
Maximum new tokens	256
Transformer blocks	8
Attention heads	8
Embedding width	384
MLP width	1,536

Table 14: Training configuration

Parameter	Value
Batch size	32
Tokens per step	8,192
Training steps	100,000
Nominal training tokens	819.2M
Optimizer	AdamW
Learning rate	3e-4
Betas	(0.9, 0.95)
Weight decay	0.1
Warmup	500 steps
Gradient clipping	1.0

Table 15: Shared data and tokenization

Item	Value
Tokenizer used to create binaries	Python tiktoken GPT-2
C++ training input	Same Python-generated train.bin and val.bin

Table 16: Training and inference throughput

Metric	C++	Python
Training throughput (tok/s)	95,000–100,000	59,000–62,000
Inference throughput median (tok/s)	190.6	129

Table 17: Recorded inference memory (median)

Metric	C++	Python
RAM (MB)	864.7	1,055
GPU memory (MB)	164.78	545

Table 18: Precision configuration

Implementation	Precision
Python	AMP/autocast + GradScaler
C++	FP32